

THOR: Lightning Detection & Alert System

Eoghan Ó Bogail



Stand number: 4201
Pobalscoil Inbhear Scéine

Contents

1	Abstract	2
2	Introduction	3
2.1	How is Lighting Detected?	3
2.2	Deciding on a Language	4
2.3	Definitions	4
3	The System	5
3.1	Architecture	5
4	Implementation	6
4.1	THOR Hub	6
4.2	Alerts App	6
4.2.1	THORICONS	7
	References	i

Comments

Chapter 1

Abstract

Chapter 2

Introduction

I live in rural Ireland, which means that I am no stranger to power cuts. A combination of wind and lightning during thunderstorms often leads to a power line or a tree falling, leaving my house without any power. These power cuts can cause small power surges to occur, which most devices aren't designed to deal with and will eventually stop working over time if the surges keep happening. If lightning strikes the house directly, say though a satellite dish, the surge would be in the millions of volts, which would absolutely kill the devices that are plugged in[2].

All of this means it is important to know if lightning is approaching. One of the simplest ways of knowing that lightning is nearby is by looking for the flash and listening for the thunder. This isn't very useful to me though, as I may not notice the flash and I tend to spend a lot of my time using a computer listening to YouTube or music, and I often don't check the weather forecast, so I often miss these cues.

Experiences like this got me thinking: what if I could get an alert on my computer when lightning strikes? Would there be any other use cases for such a system? Would I know about the lightning before I can hear the thunder?

And so I set out to make this system, with the goals of sensing lightning, processing that alert, and sending that alert out to various devices.

2.1 How is Lightning Detected?

There are several ways of detecting lightning, for example the NOAA and NASA have an instrument called the Geostationary Lightning Mapper on two of their satellites which optically detects flashes of lightning, [3] however this would be inconvenient for home use.

Another way of detecting lightning involves detecting radio frequencies (RF). When lightning strikes, it sends out a pulse of electromagnetic energy, including the light we see as a flash[1]. Networks such as ATDNET, which

was created by the Met Office in the UK and is accurate to less than 3km in western Europe, detect these pulses known as "sferics" in the frequency range of 10-14kHz (known as Very Low Frequency or VLF). It then uses triangulation to find the source of the sferic[1]. Sferics can also be picked up on simple AM radios. The lightning sensor I am using in this system, the AS3935 Franklin Lightning Sensor IC, listens to a different frequency at around 500 kHz. [4] Because this type of detection uses radio waves, detectors need to filter out interference from other devices, which may not be as big of a problem for the ATD network, but for something to be used at home its more of a concern.

2.2 Deciding on a Language

I decided to use Python as the main language for this project, simply because I'm very familiar with it. I have been using Flask, a web app framework for Python, for several years at this point, and so using Python and Flask makes building the Hub very easy for me. I didn't really consider any other languages when building this project. Of course, there is some JavaScript in the Hub for client-side code execution, but the entire rest of the project uses Python.

2.3 Definitions

Before continuing I would like to define some of the terms I will be using in this booklet.

Hub refers to both the main program controlling THOR and the hardware it is running on.

You may see **Hub server** or just **server** used to refer to the hardware THOR is running on.

You May also see **Hub software** used to refer to the program running on the server.

Node refers to hardware or software that directly gives THOR data or retrieves data from THOR.

Hardware or software that retrieves data from THOR are referred to as "receiver nodes"

Hardware or software that supplies data to THOR is referred to as "provider nodes"

Provider refers to an external source of data from outside of the THOR system, such as from a weather station or the Internet. These are fetched by the **Hub**.

Chapter 3

The System

This chapter lays out how THOR should ideally be structured.

3.1 Architecture

THOR operates on a server-client model, with the server being the Hub and the clients being the various Nodes.

The Nodes connect to the Hub over a LAN connection, or in some cases over the Internet using a VPN of some kind.

In THOR, the lightning sensor is separated from the Hub into a separate Node. This allows for more flexibility, as now the lightning sensor isn't tethered to the Hub is running on.

Nodes are encouraged to autodiscover Hubs running on the same network. This is because it isn't guaranteed that a Hub will always have the same IP address or MQTT credentials, which would be inconvenient to change on a Node using a microcontroller. Hubs should be running a UDP server that listens to port 51366. Nodes should send a UDP packet starting with `thor_node_broadcast;`, followed by an ID unique to the device. The Hub should then send a UDP packet in response.

Chapter 4

Implementation

4.1 THOR Hub

The THOR hub is the central server that the nodes and receivers connect to. It consists of three separate programs: a Python web app using Flask with a Socket.IO server, an MQTT to Socket.IO bridge, and a UDP auto-discovery server. The Flask app is able to run without MQTT support, which makes it easier to update when not connected to a network.

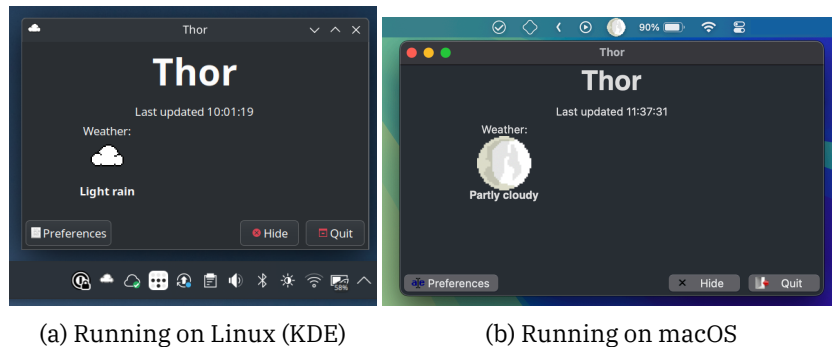
The hub receives and distributes data using the Socket.IO protocol, which is a low-latency communication protocol that mainly uses WebSockets. I picked Socket.IO because it is simple to implement on both the client and it handles reconnections automatically. The MQTT bridge allows the hub to receive alerts from Internet of Things devices.

The hub should be able to run fully on lower-powered hardware with a relatively recent Linux distribution, such as a Raspberry Pi 3B+ running Debian 12 or an old desktop, but it does require Python 3.10. It is designed to work entirely within your local network, and does not require internet access after the initial setup, however there are some optional features implemented that allows data from the internet to be used.

4.2 Alerts App

The Alerts app is a receiver node that runs on a computer. I wrote it in Python as I was already using it for the other components of this project, and I used wxPython as the GUI framework. I decided to use wxPython for various reasons, the main one being that it makes it easy to write apps that act consistently across Windows, GNU/Linux (Figure

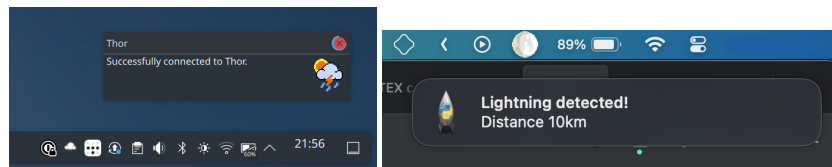
The Alerts app is very simple. It connects to THOR using Socket.IO and listens for the weather and alerts messages. When weather message arrive, it sets the system tray icon to an icon that matches the current weather.



(a) Running on Linux (KDE)

(b) Running on macOS

Figure 4.1: THOR Alerts running on various operating systems



(a) Running on Linux (KDE)

(b) Running on macOS

Figure 4.2: An notification showing an alert from THOR.

When alerts are sent out, it uses the `desktop-notifier` module to show a notification. The downsides of using `desktop-notifier` like this is that some Linux desktop environments don't support notifications and macOS won't send critical notifications (which bypass Do Not Disturb *and* Silent Mode) without a special entitlement from Apple, which requires your app is signed by Apple using a paid Developer account. Currently, the only fall-back notification type is for Windows 7 and 8.1 where it will use older "balloon" style notifications triggered through `wxPython`.

4.2.1 THORICONS

Icon sets across various systems vary wildly, especially between Linux desktop environments. This means that I would have to maintain a list of mappings from the symbol names given by MET Norway not only to Font Awesome like I previously mentioned the Hub doing, but I would also have to maintain similar lists for Microsoft's Fluent icon set, Apple's SF Symbols and the Freedesktop icon specifications (Linux), which (and this can *especially* be the case for Freedesktop-compliant icon sets) can vary wildly.

For that reason I decided to create a set of weather icons for THOR, as I am experienced in making simple pixel art drawings. The icons are made of various "parts", which are then merged into various icons by a Python script. THORICON names follow a naming convention of `part1-part2-part3`,

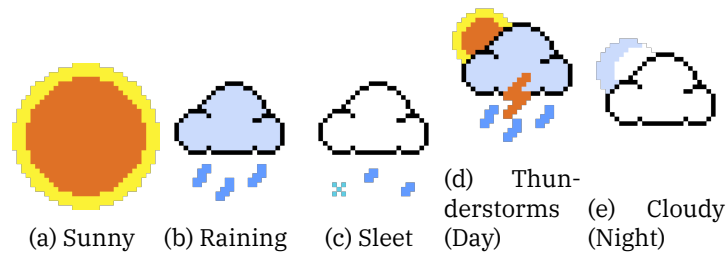


Figure 4.3: Examples of icons in the THORICON set.

which tells the script how to generate the icon. For example, when given the icon `sun-cloud-rain-lightning`, the script will paste `sun.png` onto the image, and then `cloud.png`, and so on until it has pasted all the parts in the image. It then saves the icon to `sun-cloud-rain-lightning.png`. I didn't have time to create icons for every symbol name MET Norway returns in their API, but thanks to how I generate the icons adding new icons is quick and painless.

To allow users to choose what icons they prefer on their system, THOR Alerts has a toggle in its preferences to switch between the Freedesktop and THORICON set on Linux. In the future I wish to add support for Mac and Windows icon sets directly, but I use Linux on my main computer so this is difficult to test.

References

- [1] Met Éireann. *Lightning Detection*. URL: <https://www.met.ie/science/valentia/lightning-detection>.
- [2] NOAA. *Frequently Asked Questions About Lightning*. URL: <https://www.nssl.noaa.gov/education/svrwx101/lightning/faq/>.
- [3] National Oceanic and Atmospheric Assosication. *Severe Weather 101 – Lightning Detection*. URL: <https://www.nssl.noaa.gov/education/svrwx101/lightning/detection/>.
- [4] ScioSense. *AS3935 Franklin Lightning Sensor IC Datasheet*. URL: <https://www.sciosense.com/wp-content/uploads/2023/12/AS3935-Data-Sheet.pdf>.

100% made by a human. No Generative AI was knowingly used in the creation of this booklet or this project.